

Introduction

This paper provides a collection of wording fixes to improve the precision, implementability, consistency, and completeness of the wording for defaulted comparison operator functions.

In the following areas, there was previously no specification for how a defaulted comparison operator behaves:

- when it is implicitly defined (eagerly vs lazily);
- whether it is implicitly constexpr;
- when it may be explicitly declared constexpr or consteval.

In each case, the new specification matches the behavior of defaulted special member functions to the extent possible.

The rules for when to delete a comparison function were inconsistent and varied between checks that overload resolution finds a usable function and checks that require performing full semantic analysis on the definition and determining whether it would be ill-formed. Consistent with defaulted special member functions, we now look only for surface-level problems: cases where overload resolution would fail or the structure of the class is obviously incompatible with a defaulted comparison (eg, reference members).

When determining whether a defaulted comparison is noexcept, we now consider all operations it performs, not only the invocation of comparison functions in its body. This is consistent with the behavior of defaulted special member functions, and avoids calls to terminate() if (for example) a conversion performed when initializing a parameter of a conversion function call throws an exception.

The specification for an implicitly-declared operator== was incomplete, lacking specification for various properties of the implicit declaration. Following prior CWG discussion, we inherit all properties of the operator<=> onto the operator== (other than the name and return type).

In addition, the rules for the defaulted operators other than <=> and == have been simplified and made to match the design intent, that they capture the behavior of a rewritten operator as a function whose address can be taken. For those operators, we no longer care what members the class has, since that plays no role in how the function is defaulted.

Wording

Change in 7.6.8 [expr.spaceship] paragraph 6:

If at least one of the operands is of pointer type **and the other operand is of pointer or array type**, array-to-pointer conversions (7.3.2), pointer conversions (7.3.11), ~~function pointer conversions (7.3.13)~~, and qualification conversions (7.3.5) are performed on both operands to bring them to their composite pointer type (7.2.2). ~~If at least one of the operands is of pointer-to-member type, pointer-to-member conversions (7.3.12) and qualification conversions (7.3.5) are performed on both operands to bring them to their composite pointer type (7.2.2). If both operands are null pointer constants, but not both of integer type, pointer conversions (7.3.11) are performed on both operands to bring them to their composite pointer type (7.2.2). In all cases, after~~ **After** the conversions, the operands shall have the same type. [Note: If both of the operands are arrays, array-to-pointer conversions (7.3.2) are not applied. —end note]

*Drafting note: this extends the resolution of DR 583 to three-way comparisons, and cleans up some dead wording left behind by the removal of std::*_equality. The change to disallow three-way comparisons against null pointer constants was approved by EWG as part of the discussion of P0946R0.*

Change in 10.6 [module.context] paragraph 2:

During the implicit definition of a defaulted ~~special member~~ function (11.4.3, **11.11.1 [class.compare.default]**), the instantiation context is the union of the instantiation context from the definition of the class and the instantiation context of the program construct that resulted in the implicit definition of the **defaulted special member** function.

Change in 10.6 [module.context] paragraph 4:

During the implicit instantiation of a template that is implicitly instantiated because it is referenced from within the implicit definition of a defaulted ~~special member~~ function, the instantiation context is the instantiation context of the defaulted ~~special member~~ function.

Change in 11.11.1 [class.compare.default] paragraph 1:

A defaulted comparison operator function (12.6.2) for some class C shall be a non-template function ~~declared~~ **defined** in the member-specification of C that is

- a non-static const **non-volatile** member of C having one parameter of type const C& **and either no ref-qualifier or the ref-qualifier &**, or
- a friend of C having either two parameters of type const C& or two parameters of type C.

A defaulted comparison operator function for class C that is not defined as deleted is implicitly defined when it is odr-used or needed for constant evaluation. Name lookups in the defaulted definition are performed from a context equivalent to the function-body of the defaulted operator@ function, except that the defaulted function itself is never a candidate for overload resolution.

Drafting note: the lazy definition of the defaulted function needs to perform unqualified lookup for operator functions. Such lookups should be done at the point where the function is defaulted, in a manner analogous to two-phase name lookup for templates. Lookups for a defaulted secondary operator function need to avoid finding that secondary operator function itself.

Drafting note: there is no need to require that defaulted secondary operators be defaulted in their class (or even that they have any particular parameter types). Removing such restrictions seems like a design change rather than a wording fix, so they are retained here.

Change in 11.11.1 [class.compare.default] paragraph 2:

A defaulted ~~comparison~~ **<=> or ==** operator function for class C is defined as deleted if any non-static data member of C is of reference type or C ~~is a union-like class~~ **has variant members** (11.5.1).

Drafting note: it is not necessary to consider members when defaulting a secondary comparison operator (<, <=, ...) in terms of a user-provided primary comparison operator. Also, while variant members pose a problem for defaulted primary comparisons, empty unions do not.

Add a new paragraph after 11.11.1 [class.compare.default] paragraph 2:

A defaulted comparison function is *constexpr-compatible* if it satisfies the requirements for a constexpr function ([dcl.constexpr]) and no overload resolution performed when determining whether to delete the function results in a usable candidate that is a non-constexpr function. [Note: This includes the overload resolutions performed:

- for an operator<=> whose return type is not auto, when determining whether a synthesized three-way comparison is defined,
- for an operator<=> whose return type is auto or for an operator==, for a comparison between an element of the expanded list of subobjects and itself, or
- for a secondary comparison operator @, for the expression x @ y.

—end note]

Drafting note: the implicit definition can invoke other functions, such as a conversion from some custom comparison category to the return type, or an operator@ if the use of the @ operator is rewritten in terms of a different operator. Ignoring those, and considering only the results of overload resolutions that affect deletedness, is consistent with the rule we use for defaulted special member functions. It doesn't really matter if we get this "wrong" in a corner case, since the worst case is that a comparison function is spuriously constexpr.

Change in 11.11.1 [class.compare.default] paragraph 4:

~~[...] It is unspecified whether virtual base class subobjects appear more than once in the expanded list of subobjects.~~

Add a new paragraph to the end of 11.4.3 [special]:

A defaulted special member function is *constexpr-compatible* if it the corresponding implicitly-declared special member function would be a constexpr function.

Change in 9.5.2 [dcl.fct.def.default] paragraph 3:

An explicitly-defaulted function that is not defined as deleted may be declared constexpr or consteval only if it ~~would have been~~ **is implicitly declared as** constexpr-**compatible ([special], [class.compare.default])**. If a **constexpr-compatible** function is explicitly defaulted on its first declaration, it is implicitly considered to be constexpr ~~if the implicit declaration would be~~.

Drafting note: this behavior seems unwise; consider a case such as:

```
constexpr bool operator==(const X&, const X&);
struct X { int n; friend bool operator==(const X&, const X&) = default; };
```

This program is valid, but reversing the order of the two lines renders it ill-formed because the defaulted operator== is no longer constexpr. Perhaps a better general rule would be that a constexpr-compatible defaulted function is implicitly constexpr if it is defaulted at class scope, regardless of whether that's the first declaration (that is, you always get an implicit constexpr for a defaulted comparison)? The same concern applies to the implicit exception specification. However, for a case such as:

```
struct Y { int &n; friend bool operator==(const Y&, const Y&) = default; };
bool operator==(const Y&, const Y&);
```

... the program will be ill-formed if we reverse the two lines (because the operator== is deleted after its first declaration), and there's not really anything we can do about that.

Change in 11.11.1 [class.compare.default] paragraph 3:

If the ~~class definition~~ **member-specification** does not explicitly declare ~~an == operator function~~ **any member or friend named operator==**, but declares a defaulted three-way comparison operator function, an == operator function is declared implicitly **for each defaulted three-way comparison operator function defined in the member-specification**, with the same access **and function-definition and in the same class scope** as the three-way comparison operator function, **except that the return type is replaced with bool and the declarator-id is replaced with operator==**. [Note: Such an ~~The~~ implicitly-declared == operator for a class X is an inline ~~member function~~ and is defined as defaulted in the definition of X, and has the same **parameter-declaration-clause and trailing requires-clause as the three-way comparison operator**. If the three-way comparison operator function is declared as a non-static const member, the implicitly-declared == operator function is a **non-static const member**. ~~of the form~~

```
bool X::operator==(const X&) const;
```

~~Otherwise, If the three-way comparison operator function declaration is a friend declaration,~~ the implicitly-declared == operator function is **a friend declaration**; ~~of the form~~

```
friend bool operator==(const X&, const X&);
```

~~[Note: Such~~ **such** a friend function is visible to argument-dependent lookup only. **If the three-way comparison operator function is declared virtual, constexpr, or consteval, the implicitly-declared == operator function is declared virtual, constexpr, or consteval, respectively.** —end note] **[Example:**

```
template<typename T> struct X {
    friend constexpr std::partial_ordering operator<=>(X, X) requires (sizeof(T) != 1) = default;
    // implicitly declares: friend constexpr bool operator==(X, X) requires (sizeof(T) != 1) = default;

    [[nodiscard]] virtual std::strong_ordering operator<=>(const X&) const = default;
    // implicitly declares: [[nodiscard]] virtual bool operator==(const X&) const = default;
};
```

~~—end example] The operator is a constexpr function if its definition would satisfy the requirements for a constexpr function.~~ [Note: The == operator function is declared implicitly even if the defaulted three-way comparison operator function is defined as deleted. —end note]

Drafting note: there can be more than one defaulted three-way comparison, and we previously agreed to produce a defaulted operator== for each. The old wording didn't specify many of the properties of the operator== function; inheriting them all from the operator<=> function seems like the right default behavior.

Change heading of 11.11.2 [class.eq]: Equality operators

Change in 11.11.2 [class.eq] paragraph 2:

A defaulted `==` operator function for a class `C` is defined as deleted unless, for each x_i in the expanded list of subobjects for an object `x` of type `C`, **overload resolution for $x_i == x_i$ results in a usable candidate** ~~is a valid expression and contextually convertible to `bool`.~~

Drafting note: if the return type is not contextually convertible to `bool`, this will result in a hard error.

Delete 11.11.2 [class.eq] paragraph 4:

~~A defaulted `!=` operator function for a class `C` with parameters `x` and `y` is defined as deleted if~~

- ~~• overload resolution (12.4), as applied to `x == y`, does not result in a usable function, or~~
- ~~• `x == y` is not a prvalue of type `bool`.~~

~~Otherwise, the operator function yields `!(x == y)`.~~

Change in 11.11.2 [class.eq] paragraph 5:

```
struct D {
    int i;
    friend bool operator==(const D& x, const D& y) = default; // OK, returns x.i == y.i
    bool operator!=(const D& z) const = default; // OK, returns !(*this == z)
};
```

Drafting note: `!=` is handled with all the other secondary operators.

Change in 11.11.3 [class.spaceship] paragraph 1:

[...]

- If overload resolution for `a <=> b` results in a usable **function candidate** (12.4), `static_cast<R>(a <=> b)`.
- Otherwise, if overload resolution for `a <=> b` finds at least one viable candidate, the synthesized three-way comparison is not defined.
- **Otherwise, if `R` is not a comparison category type, or if overload resolution for either the expression `a == b` or the expression `a < b` does not result in a usable candidate, the synthesized three-way comparison is not defined.**
- [...]
- Otherwise, if **(when `R` is `partial_ordering`)**, then [...]
- ~~Otherwise, the synthesized three-way comparison is not defined.~~

[Note: A synthesized three-way comparison may be ill-formed if overload resolution finds usable **functions candidates** that do not otherwise meet the requirements implied by the defined expression. —end note]

Drafting note: a built-in candidate is not a function, but must be considered here.

Change in 11.11.3 [class.spaceship] paragraph 2:

Let `R` be the declared return type of a defaulted three-way comparison operator function. ~~Given~~, **and let x_i be an expanded list of subobjects for an object `x` of type `C`, let R_i be the type of the expression $x_i <=> x_i$, or void if overload resolution applied to that expression does not find a usable function.**

- If `R` is `auto`, then **let $cv_i R_i$ be the type of the expression $x_i <=> x_i$. The operator function is defined as deleted if overload resolution applied to that expression does not find a usable candidate or if R_i is not a comparison category type ([`cmp.categories.pre`] 17.11.2.1) for any i . The** the return type is deduced as the common comparison type (see below) of `R0`, `R1`, ..., `Rn-1`. ~~If the return type is deduced as void, the operator function is defined as deleted.~~

- Otherwise, **R shall not contain a placeholder type. If** ~~if~~ the synthesized three-way comparison of type R between any objects x_i and x_i is not defined ~~or would be ill-formed~~, the operator function is defined as deleted.

Change in 11.11.3 [class.spaceship] paragraph 3:

The return value V of type R of the defaulted three-way comparison operator function with parameters x and y of the same type is determined by comparing corresponding elements x_i and y_i in the expanded lists of subobjects for x and y (in increasing index order) until the first index i where the synthesized three-way comparison of type R between x_i and y_i yields a result value v_i where $v_i \neq 0$, contextually converted to bool, yields true; V is **a copy of** v_i . If no such index exists, V is `static_cast(std::strong_ordering::equal)`.

Drafting note: the existing wording is implementable, by constructing each v_i in turn in the function's return slot, but the original design specified that a copy of the first non-zero return value is to be returned, not the value itself.

Change in 11.11.3 [class.spaceship] paragraph 4:

The common comparison type U of a possibly-empty list of n **comparison category** types $T_0, T_1, \dots, T_{n-1}$ is defined as follows:

- ~~If any T_i is not a comparison category type (17.11.2), U is void.~~
- ~~Otherwise, if~~ **If** at least one T_i is `std::partial_ordering`, U is `std::partial_ordering` (17.11.2.2).
- [...]

Drafting note: producing void here is a hack that exists to produce some answer from `std::common_comparison_category`, and should be localized to that trait.

Change heading and section label of 11.11.4 [class.~~rel~~**compare.secondary**]: ~~Relational~~ **Secondary comparison** operators

Change in 11.11.4 [class.rel] paragraph 1:

A secondary comparison operator is a relational operator (`expr.rel`) or the `!=` operator. A defaulted ~~relational~~ operator function (12.6.2) for ~~some~~ **a secondary comparison** operator @ shall have a declared return type `bool`.

Change in 11.11.4 [class.rel] paragraph 2:

The operator function with parameters x and y is defined as deleted if

- overload resolution (12.4), as applied to $x \text{ <=> } @ y$, does not result in a usable ~~function~~ **candidate**, or
- the **candidate selected by overload resolution is not a rewritten candidate** ~~operator @ cannot be applied to the return type of $x \text{ <=> } y$.~~

Otherwise, the operator function yields $x \text{ <=> } @ y \text{ @-@}$. **[Note: The defaulted operator function is not a candidate in the overload resolution for the @ operator (11.11.1 [class.compare.default]). —end note]**

Change in 12.4 [over.match] paragraph 4:

Overload resolution results in a usable ~~function~~ **candidate** if overload resolution succeeds and the selected **candidate is either not a function ([over.built]), or is a function that** is not deleted and is accessible from the context in which overload resolution was performed.

Change in 14.5 [except.spec] paragraph 11:

The exception specification for a comparison operator function (12.6.2) without a `noexcept`-specifier that is defaulted on its first declaration is potentially-throwing if and only if ~~the invocation of any comparison operator~~ **any expression** in the implicit definition is potentially-throwing.

Drafting note: the implicit definition can invoke other functions, such as a conversion from some custom comparison category to the return type. Considering those is consistent with the rule we use for defaulted special member functions.

Change in 14.5 [except.spec] paragraph 13:

An exception specification is considered to be *needed* when:

- [...]
- the exception specification is needed for a defaulted ~~special member~~ function that calls the function. [Note: ...]

The exception specification of a defaulted ~~special member~~ function is evaluated as described above only when needed; similarly, the *noexcept-specifier* of a specialization of a function template or member function of a class template is instantiated only when needed.

Change in 17.11.3 [cmp.common] paragraph 2:

Remarks: The member *typedef-name* type denotes the common comparison type (11.11.3) of $\tau_s \dots$, the expanded parameter pack, **or void if any element of τ_s is not a comparison category type**. [Note: This is ~~well-defined even~~ **`std::strong_ordering`** if the expansion is empty ~~or includes a type that is not a comparison category type~~. —end note]

Drafting note: *common_comparison_category* should probably be removed. The same functionality can be obtained using *common_type*, except the latter is more general and SFINAE-friendly.